

Modules and Component Communication

grouping your app, and making components talk



Dinesh Rawat

Instructor · Shorter Loop × Snapied

Two topics today. First, **Angular modules**: what they are, why they exist, and where they stand now. Then **component communication**: how components pass data to each other. Today is the theory and the ideas. We do the hands-on practice tomorrow.

1 What is an Angular module?

As an app grows, you get many components, and you need a way to keep them organised. An **Angular module** (written as `NgModule`) was Angular's original way to do this: a box that **groups related components, directives, and pipes together**, and declares what they need to work.

Think of a module as a folder with a label. "These components belong together, here is what they share, and here is what they let the rest of the app use." A shopping app might have a `UserModule`, a `ProductModule`, and a `CartModule`, each holding its own related pieces.

What a module holds

- **declarations** – the components, directives, and pipes that belong to this module.
- **imports** – other modules this one needs to borrow features from.
- **exports** – the pieces this module makes available to other modules.
- **providers** – the services this module shares.

2 Why modules exist: the purpose

Modules were created to solve a real problem: in a big app, the framework needs to know which components exist, what they depend on, and how to load them efficiently. Modules gave Angular that map.

- **Organisation** – group related features so the codebase is not one giant pile.
- **Reusability** – a well-made module can be reused across projects.
- **Separation of concerns** – each feature lives in its own module, independent of the others.
- **Lazy loading** – load a feature's module only when the user actually visits it, so the app starts faster.
- **Shared setup** – declare common things (like shared components or services) in one place.

3 How a module looks (so you recognise it)

A module is a class marked with the `@NgModule` decorator, similar to how a component uses `@Component`:

APP.MODULE.TS (THE OLDER MODULE STYLE)

```
import { NgModule } from "@angular/core";
import { BrowserModule } from "@angular/platform-browser";
import { AppComponent } from "./app.component";

@NgModule({
  declarations: [AppComponent], // components in this module
  imports: [BrowserModule],     // modules it needs
  bootstrap: [AppComponent],    // the root component to start with
})
export class AppModule {}
```

If you open an older Angular project, this `app.module.ts` file is the first thing that runs. It tells Angular which components exist and which one to start the app with.

4 Important: modules are now optional

Be aware of this, so you are not confused. Modern Angular has moved to **standalone components**, where each component declares what it needs on its own, and you do not need to write modules at all. Since Angular 19, new projects created with `ng new` are standalone by default and often have **no module file**.

So where does that leave modules?

- **Still fully supported** – Angular keeps modules working, so older apps run fine and you will meet them in real jobs.
- **Not the default for new code** – when you start something fresh today, you use standalone components.
- **Still useful sometimes** – mainly when using an older third-party library that is built as a module.

What to take away: understand modules so you can read and maintain the millions of existing Angular apps that use them. But for your own new work, standalone is the modern way. We teach both so nothing surprises you.

5 Component communication: the parent and child idea

Now the second topic. Real apps have many components, and they must talk to each other. When one component's tag sits inside another's template, the outer one is the **parent** and the inner one is the **child**.

Data flows two ways, each with its own tool:

Parent → Child: the parent sends data **down** using `@Input`.

Child → Parent: the child sends an event **up** using `@Output`.

6 Sending data down: @Input

The child marks a property with `@Input()`, meaning "a parent may set this":

THE CHILD

```
import { Component, Input } from "@angular/core";

@Component({
  selector: "app-child",
  template: `<h3>Hello {{ name }}</h3>`,
})
export class ChildComponent {
  @Input() name: string = "";
}
```

The parent passes a value in with square brackets:

THE PARENT

```
template: `<app-child [name]="userName"></app-child>`

// in the class:
userName: string = "Priya";
```

The child shows **Hello Priya**. The parent's `userName` flowed down into the child's `name`. Square brackets `[name]` mean "bind to a value", not the literal text.

7 Sending news up: @Output

The child raises an event with `@Output` and an `EventEmitter`, and the parent listens:

THE CHILD

```
import { Component, Output, EventEmitter } from "@angular/core";

@Component({
  selector: "app-child",
  template: `<button (click)="sendHello()">Say hello</button>`,
})
export class ChildComponent {
  @Output() helloEvent = new EventEmitter<string>();

  sendHello() {
    this.helloEvent.emit("Hello from the child!");
  }
}
```

THE PARENT

```
template: `<app-child (helloEvent)="onHello($event)"></app-child>`

onHello(text: string) {
  this.message = text; // "Hello from the child!"
}
```

Click the button, and the parent catches the event with `(helloEvent)`. The special word `$event` carries whatever the child emitted.

8 The two brackets, side by side

Syntax	Direction	Meaning
<code>[name]="value"</code>	Parent → Child	pass data down into an <code>@Input</code>
<code>(event)="handler()"</code>	Child → Parent	listen for an <code>@Output</code> event coming up

Easy way to remember: square brackets `[ ]` look like a box you put something **into** (data in). Round brackets `( )` look like an ear **listening** for something (event out).

9 Remember this

A module (NgModule) groups related components and declares what they need. Purpose: organisation, reusability, separation, lazy loading.

Modules are now optional. Modern Angular uses standalone components by default. Learn modules to read old code, use standalone for new code.

@Input sends data **down**, parent to child. Use `[prop]="value"`.

@Output sends an event **up**, child to parent. Use `(event)="handler($event)"` with an `EventEmitter`.

Brackets: `[ ]` = data in, `( )` = event out.

10 Homework (theory)

In your own words: what is an Angular module, and name two reasons modules exist.

Explain the shift: why did Angular move from modules to standalone components? One or two lines.

Direction check: for `@Input` and `@Output`, say which way the data flows and which brackets you use.

Get ready: tomorrow we build a parent and child for real, so revise these two so the hands-on part is easy.